

Implementación y Análisis Visual de Algoritmos de Ordenamiento: Merge Sort y Shell Sort

Emmanuel Jaime Jiménez, Alejandro León López, Dania Haniel Ortega Reyes,
Luis Antonio Rincón García y Diego Moisés Rojas Mata

Licenciatura en Ingeniería de Datos e Inteligencia Artificial

Universidad de Guanajuato

Salamanca, Guanajuato, México

{e.jaimejimenez, a.leonlopez, dh.ortegareyes, la.rincon.garcia, dm.rojasmata}@ugto.mx

Resumen—En el presente trabajo se aborda el desarrollo de una aplicación gráfica orientada a la visualización en tiempo real de dos algoritmos fundamentales de ordenamiento: Merge Sort y Shell Sort. Mediante una implementación en Python utilizando la biblioteca Tkinter y el manejo de hilos de ejecución, se superó el reto de animar procesos recursivos sin bloquear la interfaz. Se evaluó el rendimiento, la complejidad temporal y espacial a través de contadores de comparaciones e intercambios. Los resultados demuestran la consistencia algorítmica de Merge Sort frente a la alta eficiencia espacial de Shell Sort con secuencias de Knuth.

Index Terms—Merge Sort, Shell Sort, Tkinter, Visualización de Algoritmos, Complejidad Temporal, Python.

I. INTRODUCCIÓN

La comprensión profunda de los algoritmos de ordenamiento constituye un pilar fundamental en la formación en ciencias computacionales. Sin embargo, su análisis puramente teórico resulta insuficiente para capturar la dinámica real de su ejecución. En el presente trabajo se describe el diseño e implementación de una herramienta de visualización interactiva que permite observar en tiempo de ejecución el comportamiento de dos algoritmos de ordenamiento de relevancia histórica y técnica: Merge Sort y Shell Sort. A través de la representación gráfica de comparaciones, intercambios y profundidades de recursión, se busca tender un puente entre la notación asintótica formal y el comportamiento empírico observable [1].

I-A. Shell Sort

Shell Sort fue publicado por primera vez por Donald L. Shell en 1959 en la revista *Communications of the ACM* con el título “High-Speed Sorting Procedure” [3]. Es un algoritmo de ordenamiento *in-place* y de comparación que puede entenderse como una generalización del *insertion sort* [2]. Su idea principal consiste en ordenar pares de elementos distantes entre sí mediante un incremento (*gap*) inicial grande, reducir progresivamente dicho incremento y, cuando este llega a 1, ejecutar un *insertion sort* convencional. Esta estrategia permite mover elementos con gran desorden a su posición correcta con mayor rapidez que un intercambio de vecinos cercanos [2].

I-B. Merge Sort

Merge Sort fue desarrollado por John von Neumann en 1945 [4] y es considerado uno de los primeros algoritmos de ordenamiento importantes de la historia, concebido originalmente para la EDVAC [4]. El algoritmo sigue la técnica de divide y vencerás: si la lista contiene cero o un elemento, se considera ordenada; de lo contrario, se divide en dos sublistas de tamaño aproximadamente igual, las cuales se ordenan de forma recursiva y posteriormente se fusionan en una única lista ordenada [2, pp. 158–168]. Entre sus características principales destacan una complejidad temporal de $\mathcal{O}(n \log n)$ en todos los casos [2], la estabilidad del ordenamiento —es decir, la preservación del orden relativo de elementos iguales— y un requerimiento de $\mathcal{O}(n)$ de espacio auxiliar.

II. METODOLOGÍA

II-A. Descripción del Problema y Solución Propuesta

El desarrollo de visualizadores algorítmicos mediante interfaces gráficas de usuario (GUI) de un solo hilo, como Tkinter, presenta un problema inherente de concurrencia. La ejecución del bucle de eventos principal (*mainloop*) se bloquea transitoriamente si se ejecutan de manera síncrona rutinas iterativas densas o llamadas recursivas continuas, lo que resulta en la pérdida de responsividad del sistema y la congelación de la pantalla (*thread freezing*).

La solución arquitectónica implementada se basa en la separación estricta entre el cálculo algorítmico y el renderizado visual. Se implementó un patrón de diseño basado en la recolección de estados discretos. Los algoritmos de ordenamiento operan en memoria secundaria generando y almacenando tuplas inmutables (*snapshots*) que contienen copias profundas del arreglo, mapeos de colores y métricas acumuladas cada vez que ocurre una operación de evaluación o escritura en la estructura de datos. Una vez concluido el cálculo lógico, el renderizado de la animación se delega a un hilo de ejecución independiente (*Daemon Thread*), el cual lee secuencialmente la colección de estados, garantizando la fluidez de la interfaz gráfica y permitiendo el control asíncrono del flujo (pausa, reanudación y saltos discretos).

II-B. Explicación de la Implementación

La arquitectura del sistema se divide lógicamente en la declaración de variables de estado globales, el gestor de renderizado, las funciones de ordenamiento y los controladores de subprocesos.

II-B1. Declaración de Variables de Estado y Configuración: El estado del programa y de la interfaz gráfica está determinado por las siguientes variables globales definidas a nivel de módulo:

- **Constantes Hexadecimales:** COLOR_BASE (azul, elemento no procesado), COLOR_COMPARE (rojo, evaluación condicional), COLOR_MERGE (magenta, reubicación de memoria) y COLOR_DONE (azul claro, posición final confirmada).
- **COLOR_NIVEL:** Un arreglo de cinco cadenas hexadecimales que actúan como gradiente visual para representar la profundidad en el árbol de recursión (*Merge Sort*) o el tamaño del salto iterativo (*Shell Sort*).
- **Instancias de Tkinter:** ventana (raíz de la interfaz) y canvas (lienzo de dibujo vectorial).
- **arreglo_actual:** Lista que almacena los valores numéricos brutos generados (datos de entrada).
- **pasos:** Lista bidimensional maestra que almacena las tuplas (estado del arreglo, estado de colores, contador de comparaciones, contador de intercambios, métrica extra).
- **paso_idx:** Variable entera que actúa como puntero al índice del arreglo pasos que se está proyectando en la pantalla actual.
- **reproduciendo:** Bandera booleana (True/False) que controla la condición de paro o ejecución del hilo secundario de animación.
- **Variables de Control (Tkinter Variables):**
 - algoritmo: Instancia de tk.StringVar que almacena la selección del usuario ("merge." "shell").
 - var_tamano: Instancia de tk.IntVar vinculada al control deslizante (*slider*) del tamaño del arreglo (*n*).
 - var_velocidad: Instancia de tk.IntVar vinculada al control deslizante del retardo de la animación.
 - var_comparaciones y var_intercambios: Instancias de tk.StringVar utilizadas para renderizar dinámicamente el conteo de operaciones.
 - var_info_extra: Instancia de tk.StringVar que muestra el tamaño del gap actual, exclusivo de la ejecución de Shell Sort.

II-B2. Gestores de Renderizado Visual: dibujar(arr, colores)

- **Propósito:** Renderizar geométricamente un estado específico del arreglo en el Canvas.
- **Funcionamiento:** Primero limpia completamente la memoria gráfica (canvas.delete(.all)). Calcula el ancho de cada barra (b_w) dividiendo la resolución horizontal del lienzo entre la longitud del arreglo. Determina

la altura de cada barra calculando una razón lineal entre el valor del elemento y el valor máximo de la colección, multiplicada por la altura total del lienzo. Finalmente, traza los polígonos utilizando el método create_rectangle.

aplicar_paso(idx)

- **Propósito:** Coordinar la actualización del estado visual y las métricas de la interfaz de usuario.
- **Funcionamiento:** Verifica los límites del puntero numérico idx. Desempaqueta la tupla contenida en pasos[idx], transfiere la copia del arreglo y la lista de colores a la función dibujar(), y actualiza los valores de texto de las etiquetas mediante el método .set() de las variables var_comparaciones, var_intercambios y var_info_extra.

II-B3. Lógica de Algoritmos (Generadores de Estado): construir_merge(arr_entrada)

- **Propósito:** Generar la estructura de datos que documenta la ejecución de *Merge Sort*.
- **Funcionamiento:** Implementa un paradigma de recursión pura mediante la subrutina interna ms(l, r, depth). Manipula los índices espaciales l y r sobre una copia del arreglo maestro. La profundidad recursiva altera la indexación de los tonos visuales almacenados en COLOR_NIVEL. Utiliza una función de clausura estática llamada snap(), la cual empaqueta en una tupla las copias dinámicas del arreglo, el mapa de colores y los contadores en memoria, para finalmente anexarlas a una lista local que se retorna en bloque al finalizar la ejecución.

construir_shell(arr_entrada)

- **Propósito:** Generar la estructura de datos que documenta la ejecución de *Shell Sort*.
- **Funcionamiento:** Calcula vectorialmente la secuencia de gaps decrecientes propuesta por Knuth. Posteriormente, ejecuta la subrutina de ordenamiento por inserción iterando sobre cada partición basada en el gap en curso. Modifica condicionalmente los índices cromáticos para resaltar la región evaluada y, del mismo modo que el algoritmo anterior, inyecta llamadas constantes a snap(gap) para documentar cada iteración y asignación de memoria.

II-B4. Gestión de Concurrencia y Eventos de Usuario: iniciar(tipo=.aleatorio)

- **Propósito:** Inicializar y reiniciar el entorno de evaluación gráfica.
- **Funcionamiento:** Llama a detener() para abatir hilos activos. Evalúa el parámetro tipo para poblar la lista arreglo_actual mediante compresión de listas con enteros pseudoaleatorios, escalonados u ordenados inversamente. Deriva el flujo de control hacia la función constructora correspondiente (Merge o Shell) evaluando el objeto algoritmo.get(). Restablece el puntero

paso_idx a cero y manda a proyectar el estado estático inicial.

toggle_play()

- **Propósito:** Conmutar el ciclo de reproducción asíncrona de los estados del algoritmo.
- **Funcionamiento:** Evalúa la variable booleana reproduciendo. Si es verdadera, invoca detener(). Si es falsa, invierte la variable, desactiva el botón de avance manual, y ejecuta la instanciación de la clase threading.Thread, pasándole como objetivo la función loop_reproduccion y activando la bandera daemon=True para asegurar la finalización del hilo en caso de cierre de la ventana.

loop_reproduccion()

- **Propósito:** Bucle de iteración para la proyección cronológica en segundo plano.
- **Funcionamiento:** Implementa un ciclo while que depende lógicamente de la bandera reproduciendo y el límite del arreglo pasos. Incrementa secuencialmente paso_idx, lo envía a aplicar_paso(idx) e invoca al método de bloqueo time.sleep(), cuya latencia se recalcula en cada iteración mediante la función de decaimiento temporal obtener_delay().

obtener_delay()

- **Propósito:** Calcular algorítmicamente la latencia de hilo en milisegundos.
- **Funcionamiento:** Recupera el valor del uno al diez desde la interfaz (var_velocidad.get()) y le aplica la función $t = \max(10, 600/(v^2 \cdot 0,4 + 0,6))$, garantizando una aceleración no lineal requerida para los tiempos de visualización asintóticos, devolviendo dicho escalar flotante.

detener(), paso_adelante() y actualizar_botones()

- **Propósito:** Funciones utilitarias para la alteración de estados de control.
- **Funcionamiento:** detener() conmuta reproduciendo a falso y restaura la configuración semántica del botón btn_play. paso_adelante() realiza un incremento aritmético acotado de paso_idx (+1) e invoca al renderizador de un fotograma unitario. Finalmente, actualizar_botones() muta las propiedades state ("disabled." "normal") de los elementos interactivos evaluando de forma booleana la posición del índice relativa a la longitud del arreglo.

III. RESULTADOS

III-A. Análisis de Complejidad Temporal

Al analizar el comportamiento del código, se comprobaron las complejidades matemáticas que se resumen en la Tabla I.

Para el caso de Merge Sort, al dividir el arreglo a la mitad en cada paso recursivo, el proceso se repite sucesivamente en $\log_2 n$ niveles de división. Como en cada uno de estos

niveles se tienen que recorrer los n elementos para volver a juntarlos de forma ordenada, su tiempo de ejecución siempre es $\mathcal{O}(n \log n)$, sin importar si los números ingresan totalmente desordenados, invertidos o ya acomodados. Sin embargo, su principal desventaja es que necesita una memoria extra de $\mathcal{O}(n)$. Esto ocurre porque el algoritmo no puede ordenar todo ahí mismo; el código requiere crear listas como apoyo en cada llamada para guardar los datos mientras se fusionan.

Tabla I
ANÁLISIS DE COMPLEJIDAD ESPACIAL Y TEMPORAL

Algoritmo	Caso Promedio	Peor Caso	Espacial
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Shell Sort	$\mathcal{O}(n^{1.5})$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$

Por su parte, el Shell Sort resulta ser mucho más eficiente en cuanto al uso de memoria. Como los números se reacomodan directamente dentro del mismo arreglo usando solamente una variable sencilla para realizar los intercambios, no se gasta memoria RAM extra, por lo que su complejidad espacial es $\mathcal{O}(1)$. En cuanto a su velocidad, usar la secuencia de saltos de Knuth ayuda a evitar que el programa caiga en el peor de los casos. En la práctica, esto logra que en promedio el tiempo de ejecución ronde el $\mathcal{O}(n^{1.5})$, lo cual es una mejora enorme comparado con el tiempo $\mathcal{O}(n^2)$ de un ordenamiento clásico. Esto se debe a que los saltos grandes permiten que los números pequeños viajen rápidamente al inicio del arreglo, sin tener que moverse de un lugar a otro casilla por casilla.

IV. ANÁLISIS Y DISCUSIÓN

Al analizar los contadores de comparaciones e intercambios en el programa, se encontraron diferencias importantes entre el comportamiento de los dos algoritmos. Aunque Merge Sort mantiene un tiempo de ejecución muy estable, presenta un riesgo claro cuando se intenta ordenar una cantidad masiva de datos. Como el algoritmo funciona mediante recursión, cada vez que se divide el arreglo se hace una nueva llamada a la función, la cual se acumula en la memoria del sistema. Si el arreglo inicial es demasiado grande, se puede superar el límite permitido por el intérprete de Python, haciendo que el programa colapse con un error de desbordamiento (*RecursionError* o *Stack Overflow*). Este es un límite de software real que se debe considerar al programar, más allá de lo que dice la teoría matemática.

Por otro lado, la implementación de Shell Sort evita este problema por completo al usar solamente ciclos iterativos anidados en lugar de funciones recursivas. Al observar la interfaz gráfica durante la ejecución, se hizo evidente que los saltos amplios (*gaps*) del inicio son fundamentales, ya que logran mover los valores más grandes y más pequeños a sus respectivos extremos en muy pocos pasos. Este acomodo previo deja el arreglo prácticamente ordenado para cuando el salto finalmente se reduce a 1. Como resultado, la última pasada del algoritmo requiere una cantidad mínima de movimientos para terminar el trabajo, lo que confirma que la secuencia de

saltos es una optimización excelente en la práctica para ahorrar procesamiento.

V. CAPTURAS DE PANTALLA DE LA IMPLEMENTACIÓN

V-A. Visualización de Merge Sort

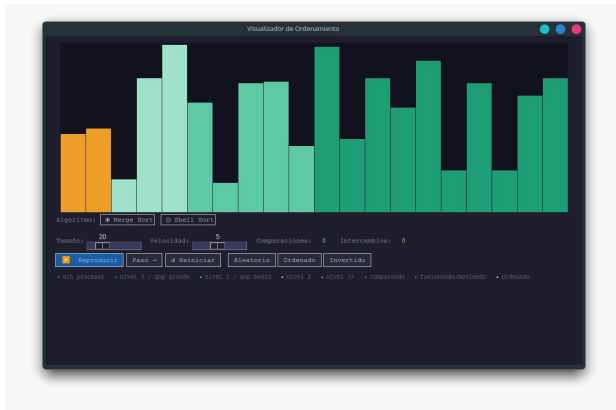


Figura 1. Profundidad máxima del árbol de recursión en *Merge Sort*: primeros elementos aislados listos para evaluación (color naranja, nivel 3+).

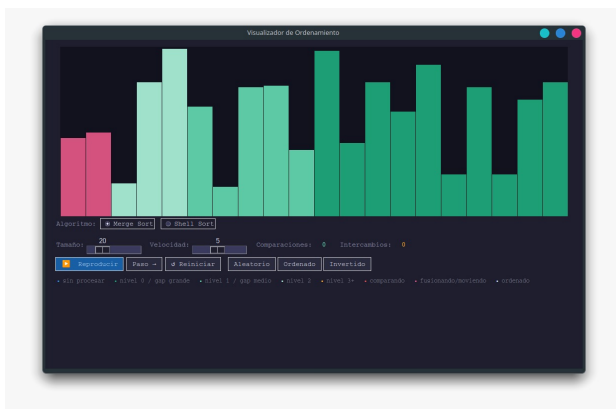


Figura 2. Fase inicial de mezcla (*Merge*). Reubicación en memoria del primer par de elementos base (color magenta).

V-B. Visualización de Shell Sort

VI. CONCLUSIONES Y APRENDIZAJES OBTENIDOS

A través del desarrollo de este proyecto, se logró la implementación exitosa de dos algoritmos de ordenamiento fundamentales, integrando tanto su lógica matemática como su representación visual. Se concluye que, si bien la recursión en Merge Sort ofrece una estructura elegante y predecible en términos de rendimiento con una complejidad $O(n \log n)$, la implementación iterativa de Shell Sort representa una solución más robusta para entornos con restricciones de memoria, gracias a su complejidad espacial constante $O(1)$.

El reto técnico más grande fue evitar que la interfaz se congelara mientras el algoritmo trabajaba. Al principio, la ventana se bloqueaba durante los cálculos, pero esto se resolvió usando hilos separados para que el proceso de dibujo

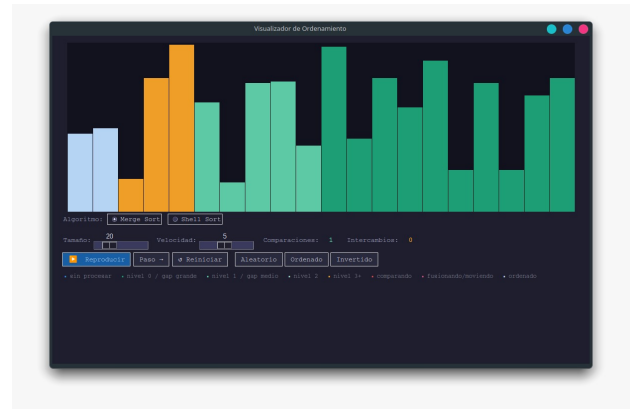


Figura 3. Consolidación del primer sub-arreglo lógico (color celeste, estado ordenado) y transición de recursión al par adyacente.



Figura 4. Fusión intermedia asimétrica: la mitad izquierda del arreglo se encuentra resuelta mientras inicia la ordenación del bloque derecho.



Figura 5. Ejecución de la subrutina final unificando las dos mitades mayores del arreglo bajo el estado de fusión.



Figura 6. Estado terminal de *Merge Sort*. Arreglo convergido en su totalidad registrando 61 comparaciones empíricas.

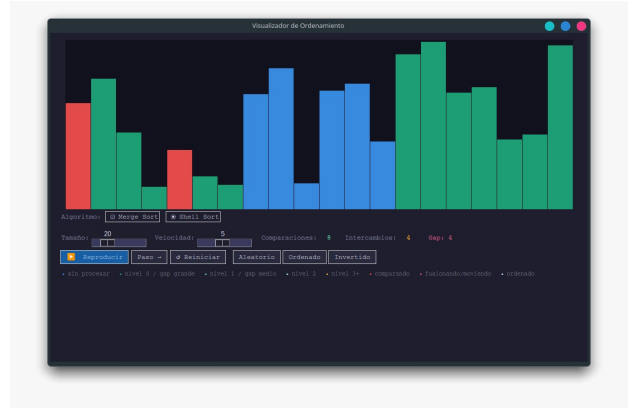


Figura 9. Reducción dinámica del incremento a $Gap = 4$ según la sucesión de Knuth, aumentando la resolución de la evaluación local.

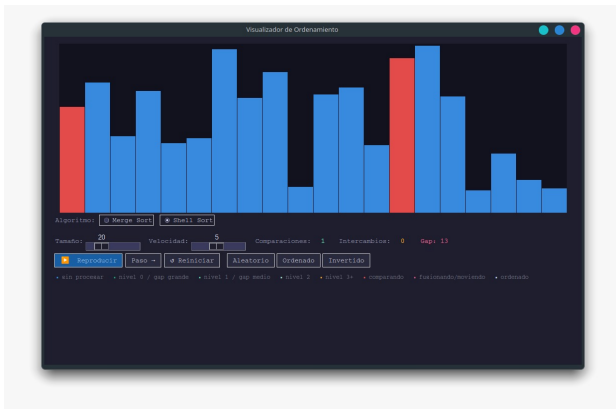


Figura 7. Fase inicial de *Shell Sort*. Aplicación del salto máximo ($Gap = 13$) evaluando elementos en los extremos de la muestra (barras rojas).



Figura 10. Transición terminal del algoritmo iterando con $Gap = 1$, operando lógicamente como un método de inserción sobre elementos adyacentes.

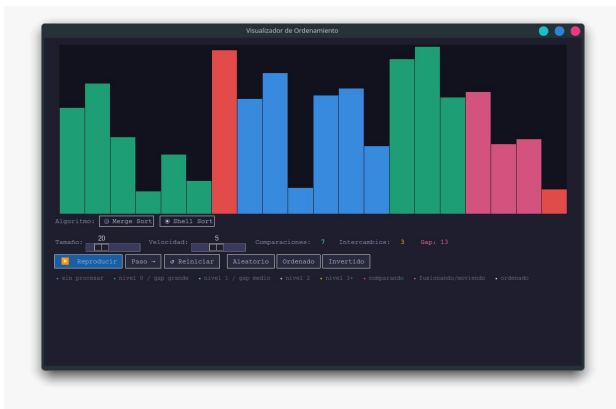


Figura 8. Intercambio de elementos bajo el gap de 13. Se observan los sub-arreglos lógicos entrelazados mediante mapeo de colores.



Figura 11. Estado terminal de *Shell Sort*. Finalización de la reubicación espacial de forma in-place.

no interrumpiera la lógica del ordenamiento. Además, durante la depuración del código, se aprendió que documentar y usar nombres claros para las variables es fundamental; organizar el sistema en módulos pequeños hizo que fuera mucho más fácil encontrar y arreglar errores. Al final, este trabajo demostró que ver en pantalla cómo se comportan los algoritmos ayuda a entender conceptos que antes solo se veían como fórmulas abstractas en papel.

REFERENCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009, pp. 170–179.
- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998, pp. 83–95.
- [3] D. L. Shell, “A high-speed sorting procedure,” *Commun. ACM*, vol. 2, no. 7, pp. 30–32, jul. 1959.
- [4] J. von Neumann, “First draft of a report on the EDVAC,” Moore School of Electrical Engineering, Univ. of Pennsylvania, Philadelphia, PA, USA, Tech. Rep., 1945.